

Preliminaries

Chapter: 2

Notations and Functions

Floor and Ceiling Functions

Let x be any real number. Then x lies between two integers called the floor and the ceiling of x . Specifically,

$\lfloor x \rfloor$, called the *floor* of x , denotes the greatest integer that does not exceed x .

$\lceil x \rceil$, called the *ceiling* of x , denotes the least integer that is not less than x .

If x is itself an integer, then $\lfloor x \rfloor = \lceil x \rceil$; otherwise $\lfloor x \rfloor + 1 = \lceil x \rceil$.

Example 2.1

$$\lfloor 3.14 \rfloor = 3, \quad \lfloor \sqrt{5} \rfloor = 2, \quad \lfloor -8.5 \rfloor = -9, \quad \lfloor 7 \rfloor = 7$$

$$\lceil 3.14 \rceil = 4, \quad \lceil \sqrt{5} \rceil = 3, \quad \lceil -8.5 \rceil = -8, \quad \lceil 7 \rceil = 7$$

Remainder Function

Let k be any integer and let M be a positive integer. Then

$$k \pmod{M}$$

(read k modulo M) will denote the integer remainder when k is divided by M . More exactly, $k \pmod{M}$ is the unique integer r such that

$$k = Mq + r \quad \text{where} \quad 0 \leq r < M$$

When k is positive, simply divide k by M to obtain the remainder r . Thus

$$25 \pmod{7} = 4, \quad 25 \pmod{5} = 0, \quad 35 \pmod{11} = 2, \quad 3 \pmod{8} = 3$$

Modulus of Negative Numbers

Problem: What is $-7 \bmod 5$?

Solution: To find $-7 \bmod 5$, we firstly find the largest number that is less than or equal to -7 and divisible by 5 . That number is -10 . Now, in order to get remainder subtract -10 from -7 , which gives 3 that is the remainder.

Thus the least positive integer that should be subtracted from -7 to make it divisible by 5 is 3 .

Integer and Absolute Value Functions

Let x be any real number. The *integer value* of x , written $\text{INT}(x)$, converts x into an integer by deleting (truncating) the fractional part of the number. Thus

$$\text{INT}(3.14) = 3, \quad \text{INT}(\sqrt{5}) = 2, \quad \text{INT}(-8.5) = -8, \quad \text{INT}(7) = 7$$

Observe that $\text{INT}(x) = \lfloor x \rfloor$ or $\text{INT}(x) = \lceil x \rceil$ according to whether x is positive or negative.

The *absolute value* of the real number x , written $\text{ABS}(x)$ or $|x|$, is defined as the greater of x or $-x$. Hence $\text{ABS}(0) = 0$, and, for $x \neq 0$, $\text{ABS}(x) = x$ or $\text{ABS}(x) = -x$, depending on whether x is positive or negative. Thus

$$|-15| = 15, \quad |7| = 7, \quad |-3.33| = 3.33, \quad |4.44| = 4.44, \quad |-0.075| = 0.075$$

We note that $|x| = |-x|$ and, for $x \neq 0$, $|x|$ is positive.

Summation Symbol; Sums

Here we introduce the summation symbol Σ (the Greek letter sigma). Consider a sequence $a_1, a_2, a_3, \dots$. Then the sums

$$a_1 + a_2 + \cdots + a_n \quad \text{and} \quad a_m + a_{m+1} + \cdots + a_n$$

will be denoted, respectively, by

$$\sum_{j=1}^n a_j \quad \text{and} \quad \sum_{j=m}^n a_j$$

The letter j in the above expressions is called a *dummy index* or *dummy variable*. Other letters frequently used as dummy variables are i, k, s and t .

Example 2.2

$$\sum_{i=1}^n a_i b_i = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n$$

$$\sum_{j=2}^5 j^2 = 2^2 + 3^2 + 4^2 + 5^2 = 4 + 9 + 16 + 25 = 54$$

$$\sum_{j=1}^n j = 1 + 2 + \cdots + n$$

Example 2.2

$$\sum_{i=1}^n a_i b_i = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n$$

$$\sum_{j=2}^5 j^2 = 2^2 + 3^2 + 4^2 + 5^2 = 4 + 9 + 16 + 25 = 54$$

$$\sum_{j=1}^n j = 1 + 2 + \cdots + n$$

The last sum in Example 2.2 will appear very often. It has the value $n(n+1)/2$. That is,

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

Thus, for example,

$$1 + 2 + \cdots + 50 = \frac{50(51)}{2} = 1275$$

Factorial Function

The product of the positive integers from 1 to n , inclusive, is denoted by $n!$ (read “ n factorial”). That is,

$$n! = 1 \cdot 2 \cdot 3 \cdots (n-2)(n-1)n$$

It is also convenient to define $0! = 1$.

Example 2.3

(a) $2! = 1 \cdot 2 = 2$; $3! = 1 \cdot 2 \cdot 3 = 6$; $4! = 1 \cdot 2 \cdot 3 \cdot 4 = 24$

(b) For $n > 1$, we have $n! = n \cdot (n-1)!$. Hence

$$5! = 5 \cdot 4! = 5 \cdot 24 = 120; \quad 6! = 6 \cdot 5! = 6 \cdot 120 = 720$$

Permutations

A *permutation* of a set of n elements is an arrangement of the elements in a given order. For example, the permutations of the set consisting of the elements a, b, c are as follows:

$$abc, acb, bac, bca, cab, cba$$

One can prove: *There are $n!$ permutations of a set of n elements.* Accordingly, there are $4! = 24$ permutations of a set with 4 elements, $5! = 120$ permutations of a set with 5 elements, and so on.

Exponents and Logarithms

Recall the following definitions for integer exponents (where m is a positive integer):

$$a^m = a \cdot a \cdots a \text{ (} m \text{ times)}, \quad a^0 = 1, \quad a^{-m} = \frac{1}{a^m}$$

Exponents are extended to include all rational numbers by defining, for any rational number m/n ,

$$a^{m/n} = \sqrt[n]{a^m} = \left(\sqrt[n]{a}\right)^m$$

For example,

$$2^4 = 16, \quad 2^{-4} = \frac{1}{2^4} = \frac{1}{16}, \quad 125^{2/3} = 5^2 = 25$$

Logarithms are related to exponents as follows. Let b be a positive number. The logarithm of any positive number x to the base b , written

$$\log_b x$$

represents the exponent to which b must be raised to obtain x . That is,

$$y = \log_b x \quad \text{and} \quad b^y = x$$

are equivalent statements. Accordingly,

$$\begin{array}{llllll} \log_2 8 = 3 & \text{since} & 2^3 = 8; & \log_{10} 100 = 2 & \text{since} & 10^2 = 100 \\ \log_2 64 = 6 & \text{since} & 2^6 = 64; & \log_{10} 0.001 = -3 & \text{since} & 10^{-3} = 0.001 \end{array}$$

Furthermore, for any base b ,

$$\begin{array}{ll} \log_b 1 = 0 & \text{since } b^0 = 1 \\ \log_b b = 1 & \text{since } b^1 = b \end{array}$$

Control Structures

2.4 CONTROL STRUCTURES

Algorithms and their equivalent computer programs are more easily understood if they mainly use self-contained modules and three types of logic, or flow of control, called

1. Sequence logic, or sequential flow
2. Selection logic, or conditional flow
3. Iteration logic, or repetitive flow

Sequence Logic (Sequential Flow)

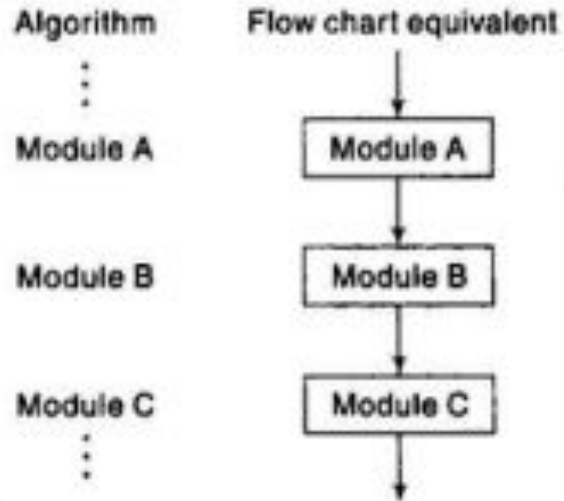


Fig. 2.3

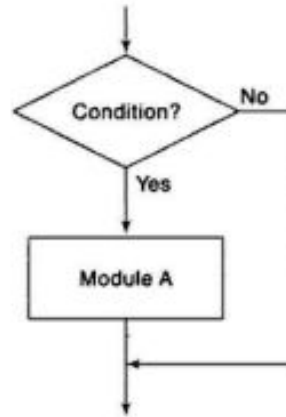
Sequence Logic

Selection Logic (Conditional Flow)

1. **Single Alternative.** This structure has the form

If condition, then:
[Module A]
[End of If structure.]

The logic of this structure is pictured in Fig. 2.4(a). If the condition holds, then Module A, which may consist of one or more statements, is executed; otherwise Module A is skipped and control transfers to the next step of the algorithm.



(a) Single alternative.

Selection Logic (Conditional Flow)

2. **Double Alternative.** This structure has the form

If condition, then:

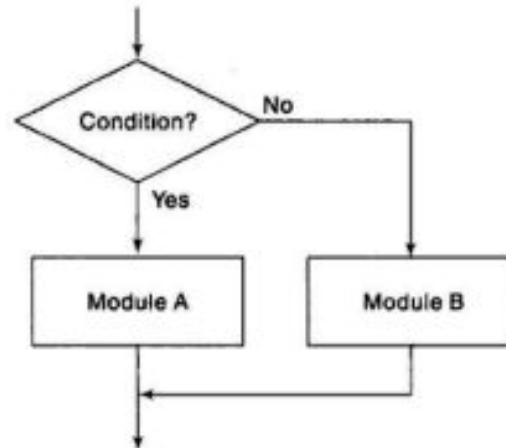
[Module A]

Else:

[Module B]

[End of If structure.]

The logic of this structure is pictured in Fig. 2.4(b). As indicated by the flow chart, if the condition holds, then Module A is executed; otherwise Module B is executed.



(b) Double alternative.

Selection Logic (Conditional Flow)

3. Multiple Alternatives. This structure has the form

If condition(1), then:

[Module A_1]

Else if condition(2), then:

[Module A_2]

$\vdots$

Else if condition(M), then:

[Module A_M]

Else:

[Module B]

[End of If structure.]

Iteration Logic (Repetitive Flow)

The logic of this structure is pictured in Fig. 2.5(a). Here R is called the *initial value*, S the *end value* or *test value*, and T the *increment*. Observe that the body of the loop is executed first with $K = R$, then with $K = R + T$, then with $K = R + 2T$, and so on. The cycling ends when $K > S$. The flow chart assumes that the increment T is positive; if T is negative, so that K decreases in value, then the cycling ends when $K < S$.

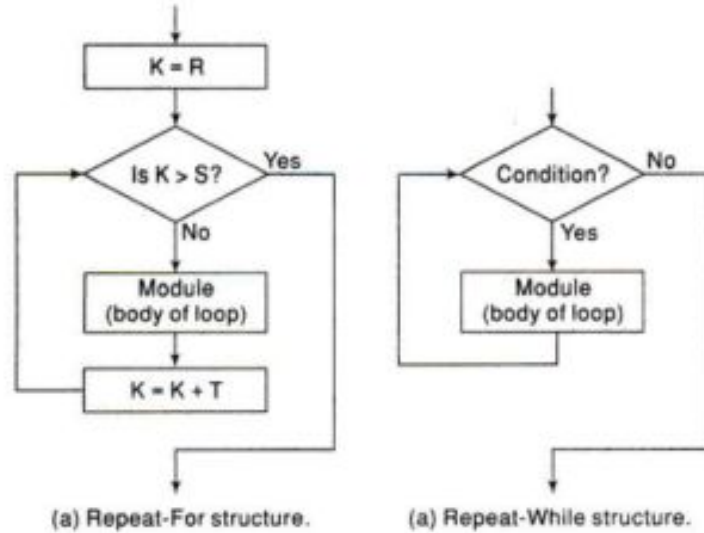


Fig. 2.5

Algorithm and Program Design

Steps in Algorithm and Program Design

1. Understand the problem
2. Design the algorithm (e.g. using pseudo codes and flowcharts)
3. Choose the most appropriate data structure
4. Translate to code
5. Testing and debugging

Local and Global concepts

Local Variable Example

```
#include <stdio.h>

void printLocal() {
    int localVar = 5; // Local variable
    printf("Local variable: %d\n", localVar);
}

int main() {
    printLocal(); // Access local variable within the function
    return 0;
}
```

Global Variable Example

```
#include <stdio.h>

int globalVar = 10; // Global variable

void printGlobal() {
    printf("Global variable: %d\n", globalVar);
}

int main() {
    printGlobal(); // Access global variable
    return 0;
}
```

Local and Global Variables

1. Global Variables:

- Useful when data needs to be shared across multiple functions or parts of the program.

1. Local Variables:

- Help encapsulate data within specific functions, preventing accidental modifications from other parts of the program.
- Encourage modularity, making the program easier to maintain and debug.

Subalgorithms

Subalgorithms

- A subalgorithm is a complete and independently defined algorithmic module which is used by some main algorithm or some other subalgorithm.
- It receives values (arguments) from the calling algorithm; performs computations and then sends back the results to the calling algorithm.

Subalgorithm Example

The following C program uses the MEAN function to calculate the average AVE of three numbers A, B and C:

Program 2.5

```
#include <stdio.h>
#include <conio.h>

void main()
{
    int A,B,C;
    float MEAN(int,int,int);
    clrscr();

    printf("Enter the values of A, B and C: ");
    scanf("%d %d %d",&A,&B,&C);

    printf("The average of %d, %d and %d is: %.2f",A,B,C,MEAN(A,B,C));
    getch();
}

float MEAN(int T1,int T2,int T3)
{
    float AVE;
    AVE=(T1+T2+T3)/3;
    return(AVE);
}
```

Output:

```
Enter the values of A, B and C: 22
36
8
The average of 22, 36 and 8 is: 22.00
```

References

- Data Structures by Seymour Lipschutz [Schaum's Outline]
- An Introduction to Data Structures with Applications by Tremblay and Sorenson

The End